SparsePipe

Co-Adaptive Sparse Collaborative LLM Inference

Treating sparsity as a time-varying scheduling resource across a heterogeneous device pipeline

Existing systems schedule *dense* computation, and existing sparsity/KV-compression methods fix one operating point on *one* device. SparsePipe makes the compression level a *control variable* chosen per pipeline boundary and per decoding step — jointly with the layer partition — under a live bandwidth, load, and accuracy budget, with a dense fallback.

This document gives the motivation, the related work with references, the method, an explicit comparison with extreme single-device KV-cache compression (Lexico, KIVI, H2O, Palu, ShadowKV, . . .), the theory targets, and an evaluation plan.

Research proposal • July 22, 2026

Contents

1 Motivation **2**
1.1 Three facts that create the opportunity . . . 2
1.2 The gap in one sentence . . . 2

2 Background and related work **2**
2.1 Collaborative / heterogeneous LLM inference . . . 2
2.2 Contextual activation sparsity . . . 3
2.3 Extreme KV-cache compression (the family SparsePipe is contrasted with) . . . 3

3 Method **3**
3.1 System model and notation . . . 3
3.2 Component 1 — block-sparse mask prediction at each boundary . . . 3
3.3 Component 2 — the joint scheduling problem . . . 4
3.4 Component 3 — adaptive wire format . . . 4
3.5 Component 4 — a cost model with *real* overheads . . . 4
3.6 Component 5 — lightweight online controller . . . 4
3.7 Component 6 — dense fallback and the accuracy floor . . . 4

4 Advantage over extreme single-device KV compression (Lexico et al.) **5**

5 Evaluation plan **6**

1 Motivation

Motivation in one paragraph

Token, neuron, attention-head, and key/value (KV) importance in an LLM are highly *non-uniform* and *time-varying*: they change with the prompt, the layer, and the decoding step. Collaborative-inference systems such as Jupiter [2] and HexGen [1] nonetheless schedule *dense* computation and transmit *dense* activations across the device pipeline. Meanwhile, the large literature on activation sparsity and KV-cache compression fixes a single operating point on a *single* device. Nobody treats sparsity as a *schedulable resource* that a distributed runtime can spend — more where the link is slow or the device is loaded, less where accuracy is at risk — jointly with the pipeline partition. SparsePipe fills exactly this gap.

1.1 Three facts that create the opportunity

1. **Sparsity is contextual and dynamic.** Contextual-sparsity work (e.g. Deja Vu [5]) shows that, for a given input, only a small, input-dependent subset of attention heads and MLP neurons materially affects the output at each layer — and that subset is *predictable* cheaply from the current hidden state. KV importance is similarly skewed: a few "heavy-hitter" tokens dominate attention [9], and importance narrows with depth [11].

2. **The bottleneck at the edge is the link, not the FLOPs.** In pipelined edge inference [3, 2], each stage passes an activation to the next stage over an ordinary LAN. When bandwidth is scarce or a device is thermally throttled, *what must cross the boundary* dominates latency. Sparsity that reduces transmission volume is therefore worth more, and worth *differently*, at different boundaries and moments.

3. **Heterogeneity is the norm.** Devices differ in compute, memory, power state, and link speed [1, 4]. A one-size operating point wastes strong, well-connected devices and overloads weak ones. The right sparsity level is a *per-device, per-step* decision.

1.2 The gap in one sentence

Current work chooses *either* a sparse model *or* a distributed layout; SparsePipe *co-designs* them — so the sparsity pattern, the wire format, and the layer partition are decided together, online, against a system-level objective.

2 Background and related work

2.1 Collaborative / heterogeneous LLM inference

HexGen [1] serves a 70B model across a heterogeneous, cross-datacenter GPU pool via *asymmetric parallelism* (each pipeline stage may hold a different number of layers and a different tensor-parallel degree), choosing the layout with a validated analytic cost model and a two-phase search. **Jupiter** [2] targets edge Jetson clusters, argues pipeline parallelism beats tensor/sequence parallelism on low-bandwidth LANs, and introduces intra-sequence pipelining plus speculative and outline-based decoding. The **FSU pipelined-inference poster** [3] partitions a model by layers across three Jetson Orin Nanos so each device keeps its own KV cache in fast memory, avoiding CPU↔GPU swapping (40% faster end to end). *All three schedule dense computation*; none treats sparsity as a scheduling variable. **FOCI** [4] adds max–min fairness for resource allocation across mobile devices — the source of SparsePipe's optional worst-device-latency objective term.

2.2 Contextual activation sparsity

Deja Vu [5] predicts, from the current hidden state, which heads and neurons are active and skips the rest, achieving large speedups without accuracy loss. SparsePipe adopts the *predictability* of contextual sparsity but shifts the decision from "skip to save FLOPs on one GPU" to "skip/transmit to satisfy a distributed schedule."

2.3 Extreme KV-cache compression (the family SparsePipe is contrasted with)

These reduce KV memory or attention cost *on a single device*:

- **Quantization:** KIVI [7] (tuning-free asymmetric 2-bit: per-channel keys, per-token values); Coupled Quantization [8] (couples channels to reach ∼1 bit/channel).
- **Token eviction / attention sparsity:** H2O [9] (keep recent + heavy-hitter tokens); SnapKV [10] (prefill-time selection via an observation window); PyramidKV [11] (more budget in lower layers); Quest [12] (query-aware page selection, loads only top-K critical pages).
- **Low-rank KV:** Palu [13] (low-rank projection of the KV latent, reconstruct on the fly, ∼50% compression); ShadowKV [14] (low-rank keys on GPU, values offloaded to CPU).
- **Sparse coding / dictionary: Lexico** [6] — the closest in spirit — represents each cached key/value as a *sparse* combination of atoms from a frozen *universal* overcomplete dictionary (∼4096 atoms, learned once), found by Orthogonal Matching Pursuit (OMP); stores ∼8 nonzero 8-bit coefficients per token, reaching **15–25% of the FP16 cache at 90–95% of full-cache accuracy**, degrading more gracefully than 2-bit quantization below its floor.
- **Merging:** CaM [15] fuses to-be-evicted KV into retained entries.

Section 4 makes the comparison with this family explicit; the key point is that these are *single-device compression primitives*, whereas SparsePipe is a *distributed scheduling layer* that can *invoke* them.

3 Method

3.1 System model and notation

A model of L decoder layers is partitioned into P pipeline stages placed on heterogeneous devices $\{d_1, \ldots, d_P\}$. A **partition** $\pi = (b_1, \ldots, b_{P-1})$ sets the stage boundaries (layer indices). At boundary p and decoding step t, stage p produces an activation tensor $X_{p,t} \in \mathbb{R}^{b \times h \times H}$ (batch b, heads h, hidden H) plus KV updates, and must forward what stage $p{+}1$ needs. Each device has a time-varying compute rate $c_p(t)$, memory M_p, and each link a bandwidth $B_p(t)$.

3.2 Component 1 — block-sparse mask prediction at each boundary

At boundary p, a lightweight predictor g_{θ_p} maps the current hidden state to a **hardware-friendly block-sparse mask**

$$m_{p,t} = g_{\theta_p}(X_{p,t}) \in \{0,1\}^{n_{\text{blk}}},$$

selecting which *blocks* of tokens, neurons/FFN channels, attention computations, and KV entries are retained. Block (not per-element) granularity is chosen so that the induced sparse kernels are actually fast on the target hardware and the index overhead stays small. The predictor is the distributed analogue of Deja Vu's [5] sparsity predictor.

3.3 Component 2 — the joint scheduling problem

At each step the runtime chooses, *together*: the partition π, a per-boundary token budget τ_p, the active neuron/KV blocks (via $m_{p,t}$), and the wire format f_p. Let σ denote this joint plan. The objective is multi-term:

$$\min_{\sigma} \; \alpha \underbrace{\mathrm{TTFT}(\sigma)}_{\text{prefill latency}} + \beta \underbrace{\mathrm{TPOT}(\sigma)}_{\text{time/token}} + \gamma\,\mathrm{Energy}(\sigma) + \delta\,\mathrm{Mem}(\sigma) + \epsilon \underbrace{\max_{p} \mathrm{Lat}_p(\sigma)}_{\text{worst device (opt.)}} \quad \text{s.t.} \quad \mathrm{AccErr}(\sigma) \le \rho.$$

The last constraint is an *accuracy floor* (Component 6); the worst-device term borrows FOCI's fairness view [4].

3.4 Component 3 — adaptive wire format

Transmission volume is not just a function of how many values survive; it is *format-dependent*. SparsePipe switches format by the realized sparsity level s_p:

$$f_p = \begin{cases} \text{dense tensor}, & s_p \text{ low (index overhead not worth it)}, \\ \text{bitmap / block-index + values}, & s_p \text{ high}. \end{cases}$$

The crossover is where the index/bitmap bytes equal the values saved — computed from the block size and the current s_p, not assumed.

3.5 Component 4 — a cost model with *real* overheads

The scheduler predicts latency/energy from measured *sparse-kernel throughput* and *index/bitmap overhead*, not idealized FLOP counts:

$$\mathrm{Lat}_p(\sigma) = \underbrace{T_p^{\mathrm{comp}}(m_{p,t}, c_p(t))}_{\text{profiled sparse-kernel time}} + \underbrace{\frac{\mathrm{Bytes}(f_p, m_{p,t})}{B_p(t)}}_{\text{transmission}}.$$

Because measured kernel efficiency is inside the model, the optimizer never chooses "sparsity" that is slower in practice — the central failure mode of unstructured sparsity.

3.6 Component 5 — lightweight online controller

Bandwidth, device load, and observed sparsity drift during a generation. A small controller (bandit or short-horizon MPC; optionally the sparse successor-state model of the companion agenda) re-plans σ when the state moves, amortizing planning over many steps as in Jupiter/HexGen but reacting *online*.

3.7 Component 6 — dense fallback and the accuracy floor

For coding, mathematics, or low-confidence outputs, SparsePipe reverts to exact dense compute (mirroring Jupiter's pluggable outline decoding [2]). More generally, the floor is enforced per boundary: bounding the output perturbation induced by a mask through the layer's Lipschitz constant lets the scheduler treat accuracy as a *constraint* rather than a hope.

Theory target

Per-boundary accuracy guarantee: if layer $p{+}1{:}L$ is κ-Lipschitz and the masked activation satisfies $\|X_{p,t} - \hat{X}_{p,t}\| \le \eta_p$, then the output perturbation is at most $\kappa \prod_{q>p}(\cdot)\,\eta_p$; the scheduler admits only plans whose aggregate bound stays below ρ. Target: an end-to-end statement that SparsePipe meets a latency/energy budget at a *certified* accuracy floor, recovering dense inference when $\rho \to 0$.

4 Advantage over extreme single-device KV compression (Lexico et al.)

The whole taxonomy of §2.3 — Lexico [6], KIVI [7], H2O [9], Palu [13], ShadowKV [14], ... — compresses the KV cache *in place on one accelerator* to fit VRAM or cut attention cost. SparsePipe operates on a different axis. The contrasts below are the defensible core of the proposal.

1. **Scheduled vs. fixed operating point.** Lexico fixes the sparsity level s offline; KIVI fixes a bit-width; H2O fixes a token budget. Even the "dynamic" methods (PyramidKV across layers, Quest per query) adapt only *within one device's attention computation*. SparsePipe chooses the compression level *per boundary and per decoding step* as the output of a scheduler driven by live bandwidth, load, and an accuracy budget — the ratio is a decision, not a hyperparameter.

2. **Communication-aware vs. communication-blind.** Every method in the family optimizes memory footprint or attention latency on *one* GPU; none has a notion of an inter-device link whose bandwidth is the bottleneck. For them, sparsity never trades against *what must be transmitted*. SparsePipe makes sparsity directly reduce activation/KV *transmission volume* across the partition boundary, coupled to the measured link bandwidth.

3. **Partition- and heterogeneity-aware.** None of these methods decides where the model is cut or which device does what. SparsePipe makes the layer partition and the sparsity schedule *jointly*: a throttled device can be given a lighter, sparser share and the boundary can move — a decision that is meaningless in a single-device compressor.

4. **Accuracy floor as a lever, not a fixed loss.** For Lexico accuracy falls monotonically as s drops; for KIVI/CQ there is a hard bit-floor. The loss is a property of the fixed setting. SparsePipe ties the accuracy floor to a scheduling decision with a dense fallback, so the accuracy–latency point is chosen online and worst-case error is bounded (Component 6).

5. **Multi-modal vs. KV-only sparsity.** The family sparsifies *one* object: the KV cache. SparsePipe schedules *token, neuron/FFN, attention, and KV* sparsity together, so "drop these neurons on the weak device this step" or "send only these tokens' activations across the slow link" become expressible.

6. **Closed-loop vs. committed offline.** Lexico's dictionary is trained once offline, and KIVI, H2O, and SnapKV commit to a policy before serving. SparsePipe is an online controller reacting to per-step system state (queue depth, link, remaining budget).

Distinctive claim

SparsePipe does not compete with Lexico at Lexico's own metric — it *subsumes* it as a primitive. Any single-device compressor (Lexico, KIVI, Palu) can be the operator the scheduler invokes on the KV that must cross a boundary. The novelty is the *scheduling layer* that makes compression bandwidth-, partition-, and load-aware with a dense fallback. Positioning SparsePipe as *orthogonal and composable* is both stronger and harder to attack than claiming to beat Lexico on compression ratio.

Capability	Lexico / KIVI / H2O / Palu	SparsePipe
Compression ratio adapts per step	no (fixed offline)	**yes (scheduled)**
Aware of inter-device bandwidth	no	**yes**
Chooses layer partition / device	no	**yes**
Accuracy floor is a runtime lever	no (fixed loss)	**yes (+ dense fallback)**
Sparsity modalities	KV only	**token/neuron/attn/KV**
Online closed-loop controller	no	**yes**
Can invoke the others as a primitive	—	**yes (composable)**

Reviewer pre-emption. ShadowKV [14] is the one family member that touches the memory hierarchy (GPU↔CPU value offload), so it may be raised as prior art for "distribution." It is single-node offload with a *fixed* policy — not bandwidth-adaptive scheduling across a heterogeneous multi-device pipeline with a movable partition. Contrasts 2 and 3 are where SparsePipe's defensibility lives.

5 Evaluation plan

- **Testbed:** the 3-device Jetson Orin cluster of the poster [3] or Jupiter's Jetson setup [2]; throttle a link mid-generation to test the online controller.
- **Models / tasks:** Llama-3.2-3B / Llama-2-7B; LongBench and GSM8K (the Lexico benchmarks, for a like-for-like accuracy axis) plus WizardLM/Vicuna with GPT-judge quality.
- **Baselines:** dense pipeline (poster/Jupiter); *single-device* compressors applied per stage (Lexico, KIVI, H2O) to show SparsePipe's scheduling adds value *on top of* them; a static-sparsity ablation (fixed ratio, no controller).
- **Metrics:** TTFT, TPOT, energy, peak memory, accuracy, worst-device latency; and the fraction of steps that fall back to dense.
- **Ablations:** remove the online controller; remove adaptive wire format; remove the joint partition (fix π); KV-only vs. multi-modal sparsity.

Main risk & mitigation

Realized speedup from sparsity is often eaten by kernel and index overhead. Mitigate by committing to *block* sparsity sized to the hardware and by putting measured kernel efficiency *inside* the cost model (Component 4), so unprofitable sparsity is never selected. Second risk: predictor mispredictions violate the accuracy floor — mitigate with the Lipschitz-bounded admission test and the dense fallback.

References

[1] Y. Jiang et al. *HexGen: Generative Inference of Large Language Model over Heterogeneous Environment.* ICML 2024. arXiv:2311.11514.

[2] S. Ye et al. *Jupiter: Fast and Resource-Efficient Collaborative Inference of Generative LLMs on Edge Devices.* IEEE INFOCOM 2025. arXiv:2504.08242.

[3] H. Zhou and X. Zhang. *Poster: Distributed Pipelined Collaborative Inference for LLMs on Edge Device Clusters.* ACM HotMobile 2026.

[4] H. Zhou and X. Zhang. *Fairness-Oracular MARL with Competitor-Aware Signals for Collaborative Inference.* NeurIPS 2025 Workshop (AI4NextG).

[5] Z. Liu et al. *Deja Vu: Contextual Sparsity for Efficient LLMs at Inference Time.* ICML 2023. arXiv:2310.17157.

[6] J. Kim, J. Park, J. Cho, D. Papailiopoulos. *Lexico: Extreme KV Cache Compression via Sparse Coding over Universal Dictionaries.* ICML 2025. arXiv:2412.08890.

[7] Z. Liu et al. *KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache.* ICML 2024. arXiv:2402.02750.

[8] T. Zhang, J. Yi, Z. Xu, A. Shrivastava. *KV Cache is 1 Bit Per Channel: Efficient Large Language Model Inference with Coupled Quantization.* arXiv:2405.03917, 2024.

[9] Z. Zhang et al. *H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models.* NeurIPS 2023. arXiv:2306.14048.

[10] Y. Li et al. *SnapKV: LLM Knows What You Are Looking For Before Generation.* arXiv:2404.14469, 2024.

[11] Z. Cai et al. *PyramidKV: Dynamic KV Cache Compression based on Pyramidal Information Funneling.* arXiv:2406.02069, 2024.

[12] J. Tang et al. *Quest: Query-Aware Sparsity for Efficient Long-Context LLM Inference.* ICML 2024. arXiv:2406.10774.

[13] J. Chang et al. *Palu: Compressing KV-Cache with Low-Rank Projection.* ICLR 2025. arXiv:2407.21118.

[14] H. Sun et al. *ShadowKV: KV Cache in Shadows for High-Throughput Long-Context LLM Inference.* ICML 2025. arXiv:2410.21465.

[15] Y. Zhang et al. *CaM: Cache Merging for Memory-efficient LLMs Inference.* ICML 2024, PMLR 235:58840–58850.